

Efficient Data Storage Systems

(Sistemas de Armazenamento de Dados Eficientes)

Data Striping, Replication, and Erasure Coding



Storage Optimizations

Landscape

● Performance

- Caching (*part 1*)
- Prefetching, read-ahead and indexing
- Tiering (*part 1*)
- I/O control policies (*part 1, annex*)
- RAID 0
- Data striping/sharding (*part 3*)

● Space management

- Compression (*part 2*)
- Deduplication (*part 2*)

● Security and privacy

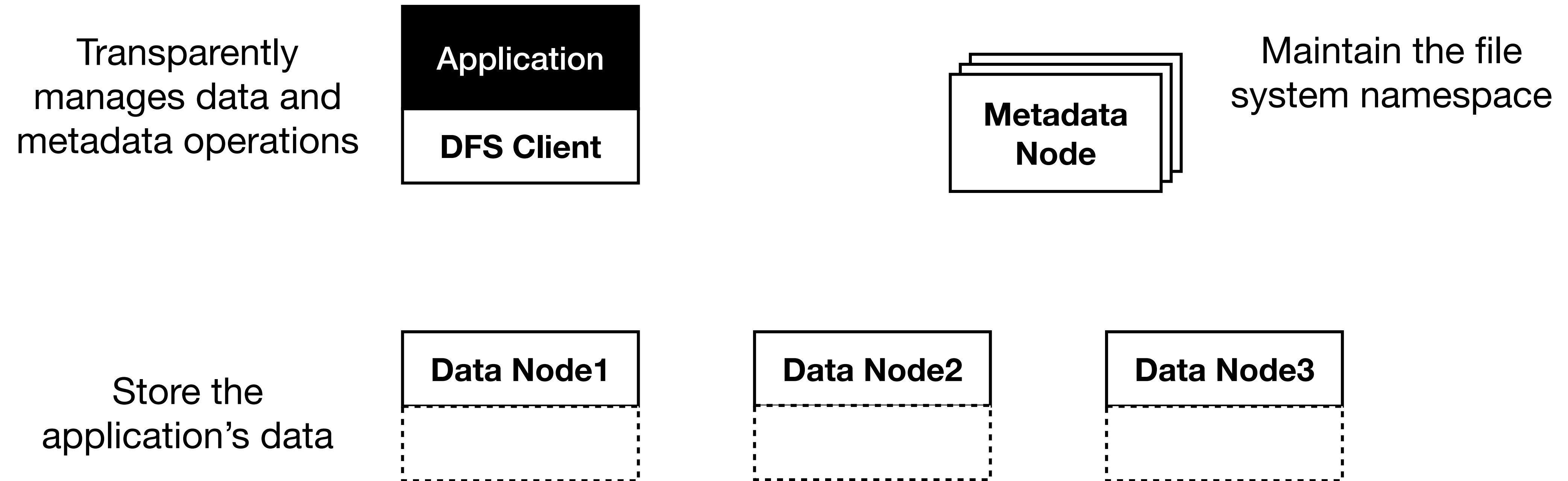
- Encryption (*part 2*)
- Access control
- Trusted execution environments

● Redundancy and crash consistency

- Replication and erasure coding (*part 3*)
- RAID
- Journaling and write-ahead logging
- Checksumming
- **And many more!**

Data management

General architecture



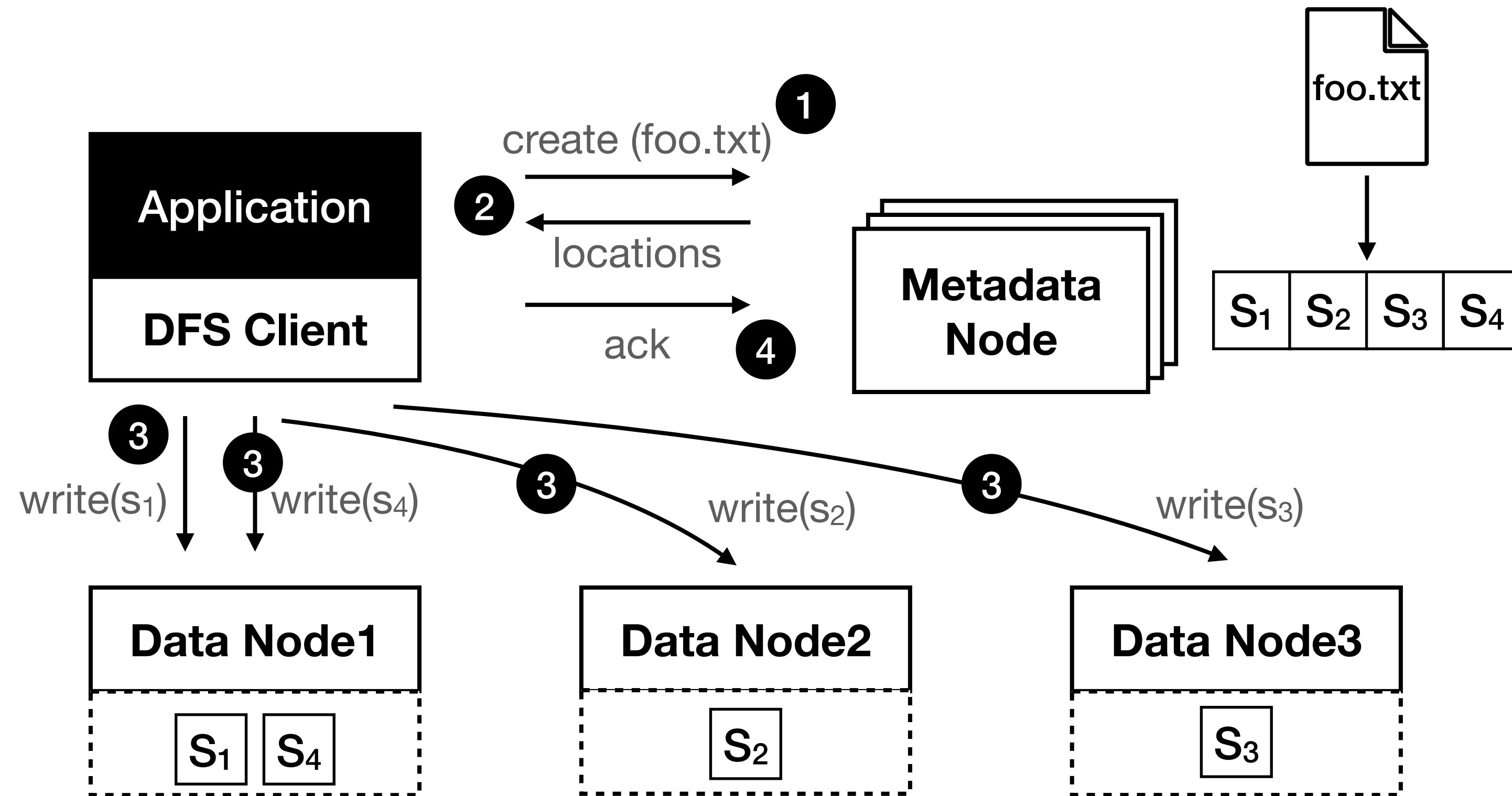
Data management

Data striping

● Data is sequentially split across nodes (or disks) to improve performance

- ▶ Files are partitioned into fixed-sized chunks (blocks or stripes)
- ▶ Each chunk is stored on a different node
- ▶ I/O operations can occur in parallel across multiple nodes

● Used in RAID 0, Lustre, GPFS, and more



Data management

Data striping: distribution strategies and tuning

- **Data distribution** determines the layout of data chunks across nodes
 - **Round-robin:** data chunks are distributed across nodes sequentially in a cyclic manner
 - **Hashing:** data chunk ID is used by a hash function to calculate the corresponding storage node ID
 - **Dynamic load-aware:** monitors real-time load status of each storage node (throughput, outstanding I/O requests, etc) and directs data chunks to nodes with lower load
- Some file systems expose tuning knobs to improve performance
 - **Stripe size:** size of a stripe unit (chunk). Large stripes are suited for sequential access to large files, while small stripes are more efficient for random access to small files
 - **Stripe width:** number of nodes involved in the striping process (parallelism level)

Data management

Data striping: summary

- **High throughput** due to the support of parallel reads and writes
- Natural **load balancing** due to distributed data access across nodes
- **No redundancy**: a failure in one node results in data loss, unless combined with replication or erasure coding
- **Increased network overhead** if data is striped across (physically) distant nodes

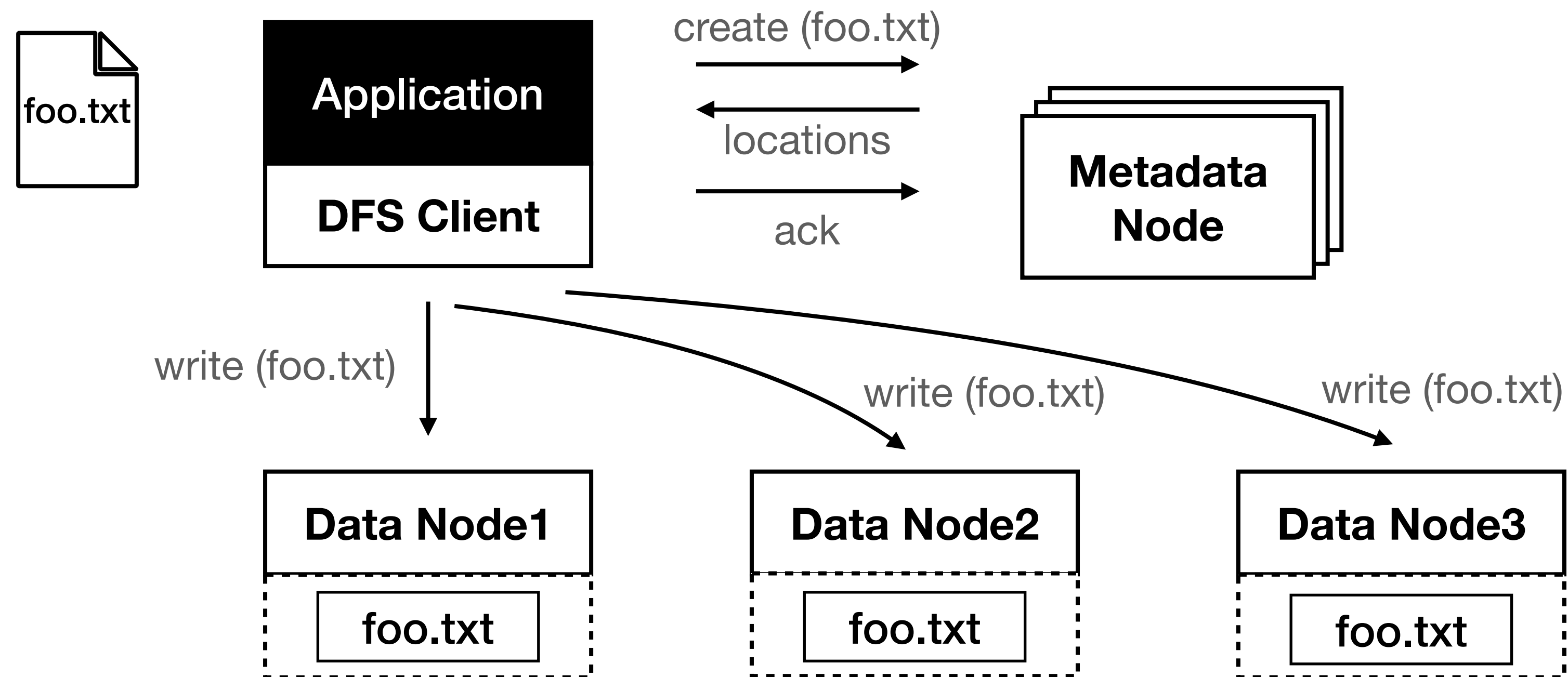
Data redundancy

Data replication

- Create multiple copies of data across different nodes
- Usually done at the file or block level
 - **High availability:** ensures access even if some node fails
 - **Fault tolerance:** reduces the risk of data loss
 - *For N replicas, tolerates $N-1$ faults*
 - **Load balancing:** distributed read requests to improve performance
 - *Some implementations are sensitive to the number of readers*
 - **Storage overhead:** more replicas require more space ...

Data replication

General workflow



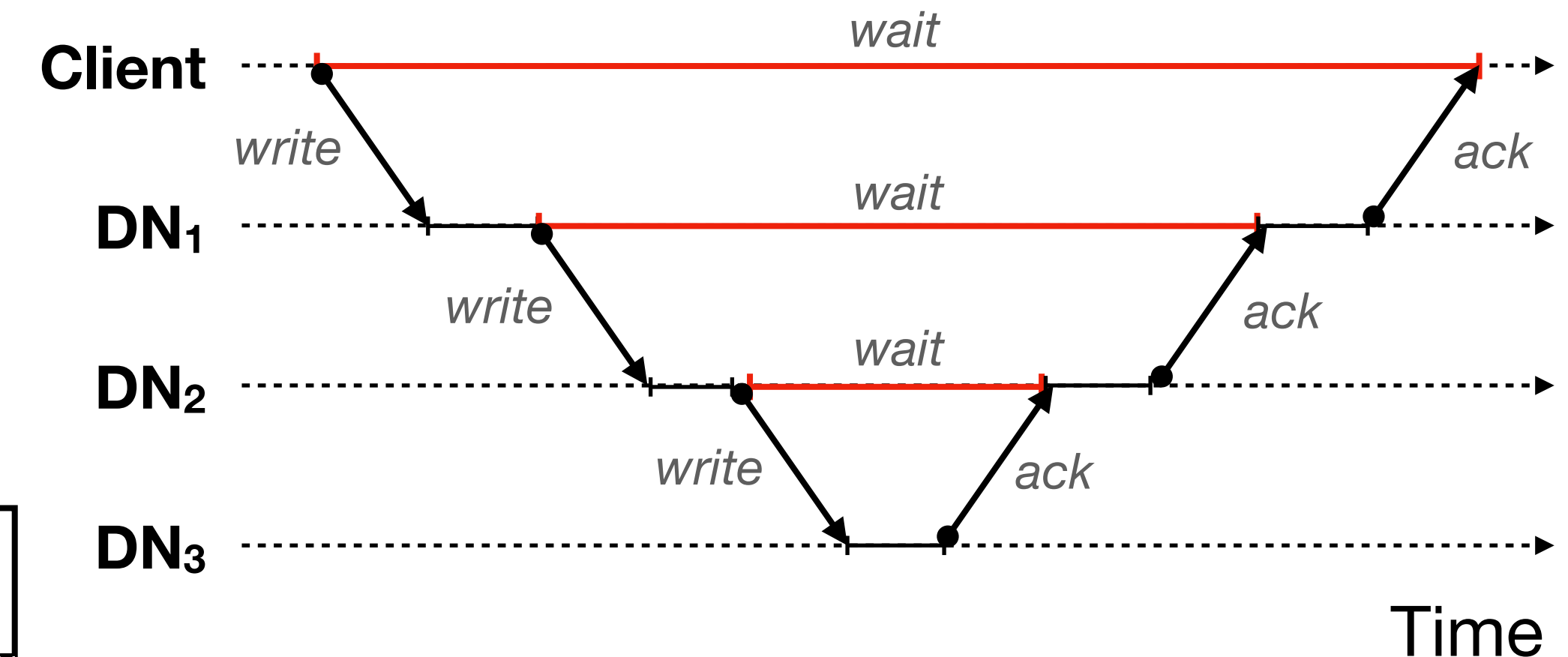
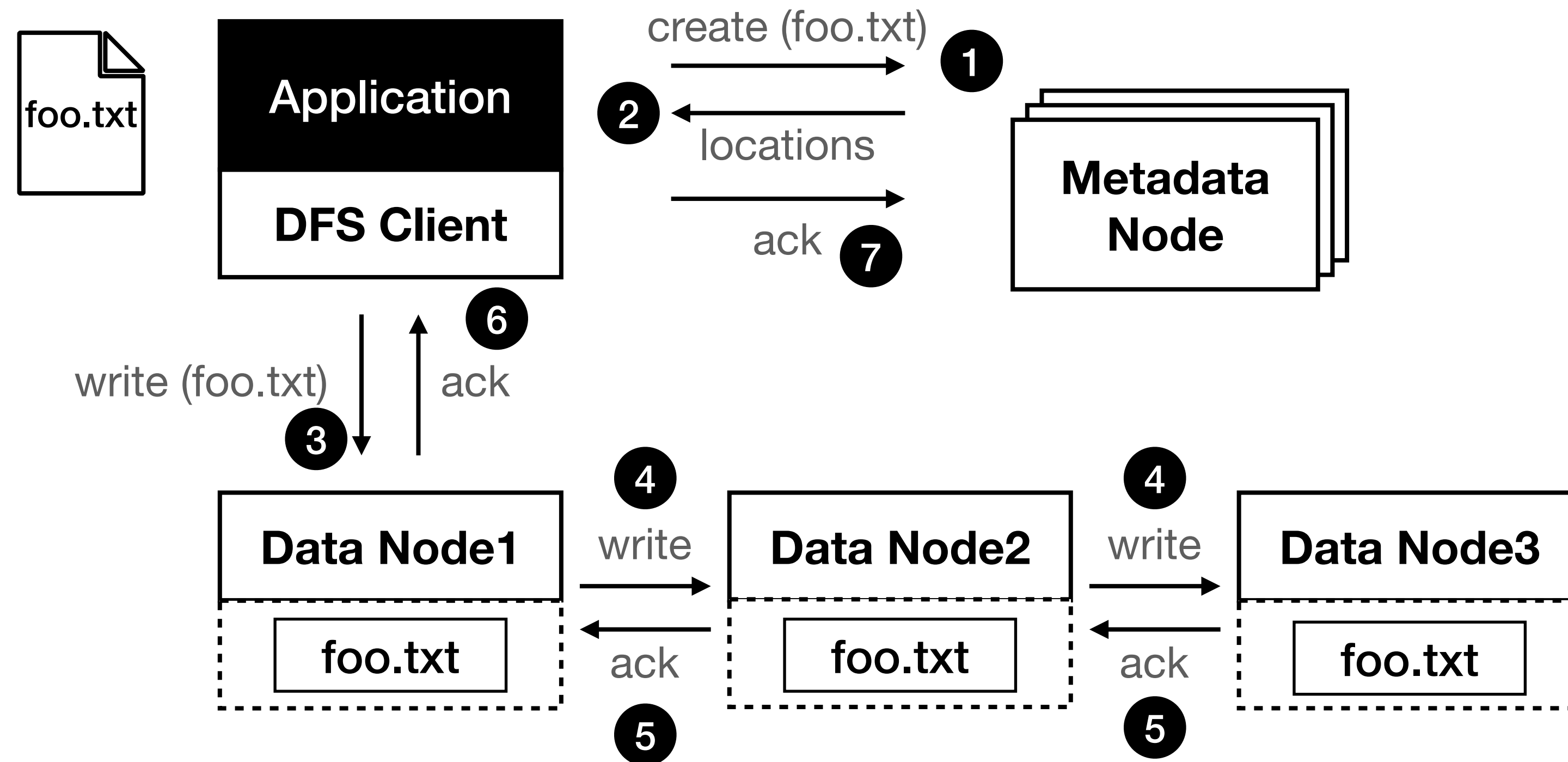
Data replication

Replication strategies

- There are several strategies for data replication
 - Synchronous replication vs. asynchronous replication
 - Passive replication vs. active replication
 - Geo-replication
 - And more

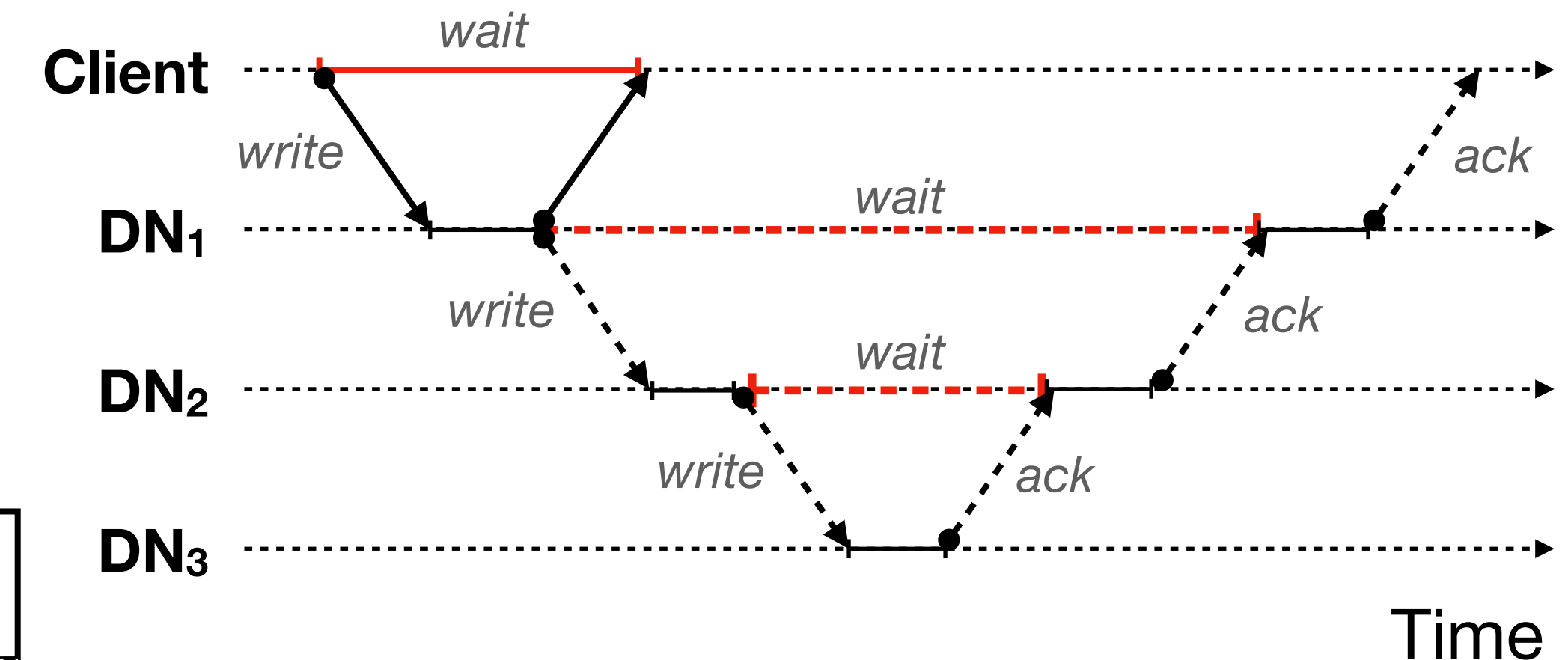
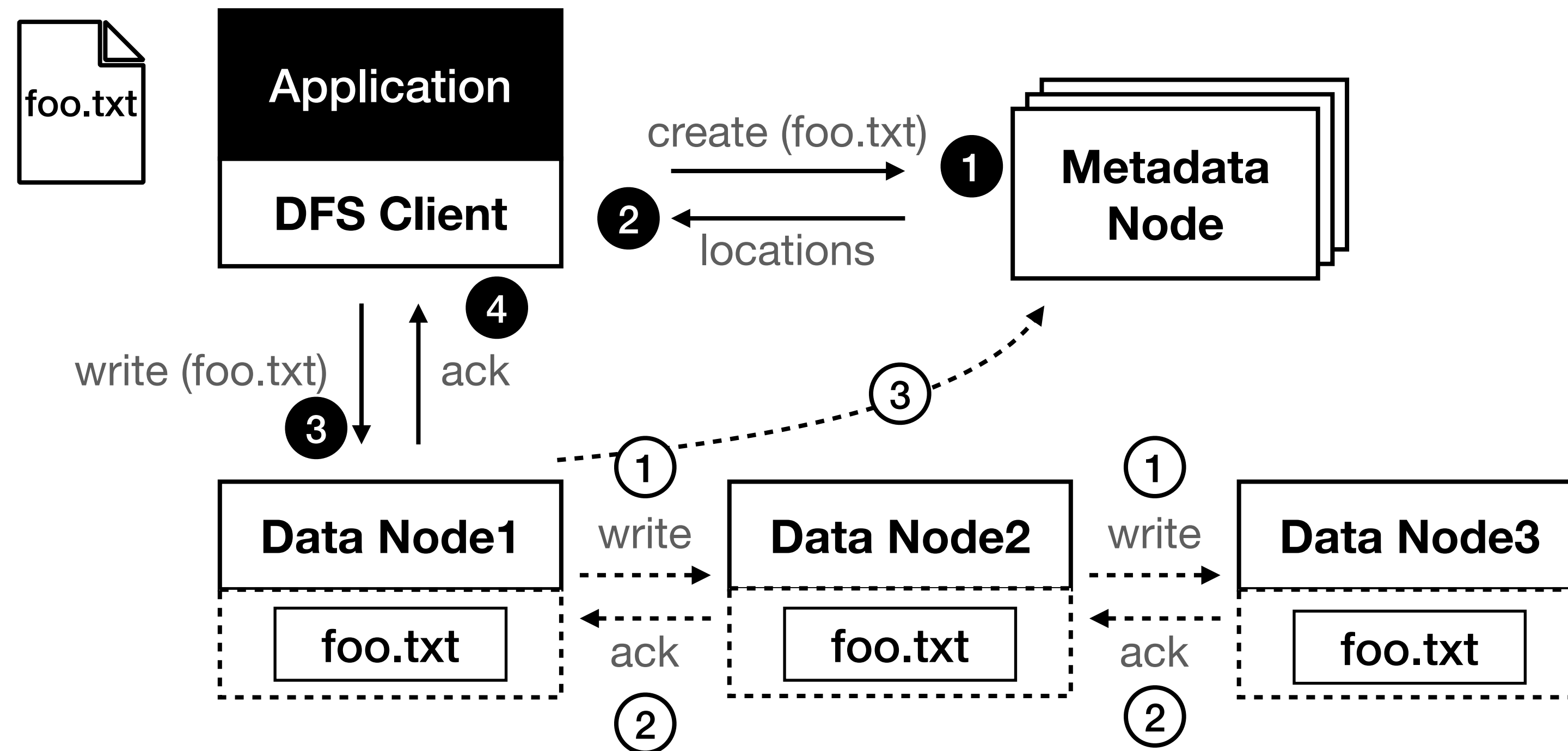
Data replication

Replication strategies: synchronous replication



Data replication

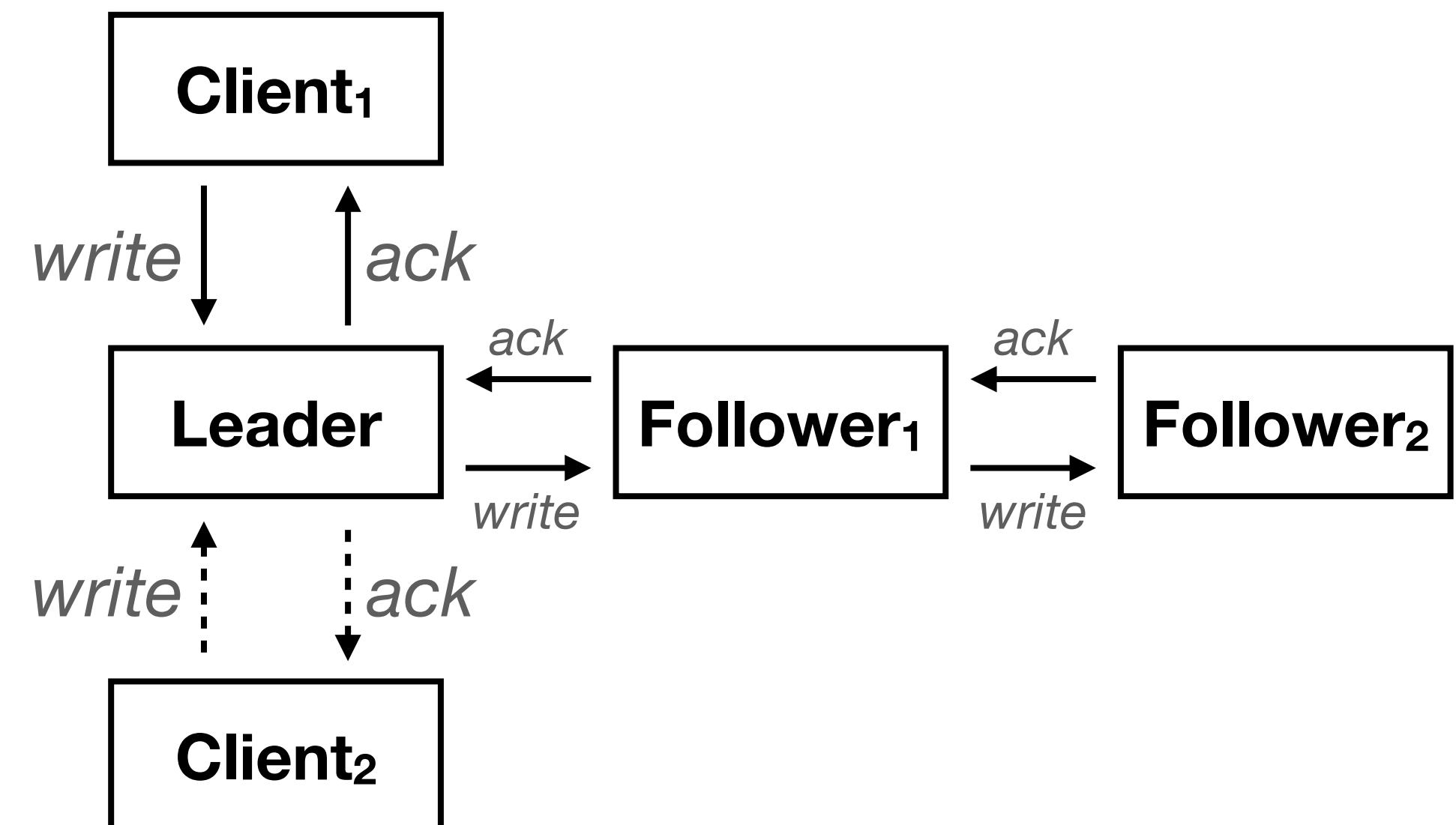
Replication strategies: asynchronous replication



Data replication

Replication strategies: passive replication

- A single node (*leader*) processes all client requests, while one or more backup nodes (*followers*) maintain a synchronized copy of the system
- Also known as *primary-backup replication* and *active-passive replication*
- Client sends all requests to the *leader*
- The leader processes the requests and updates its state
- The leader sends the updated state to the followers
- The leader sends the response back to the client once all servers have a replica



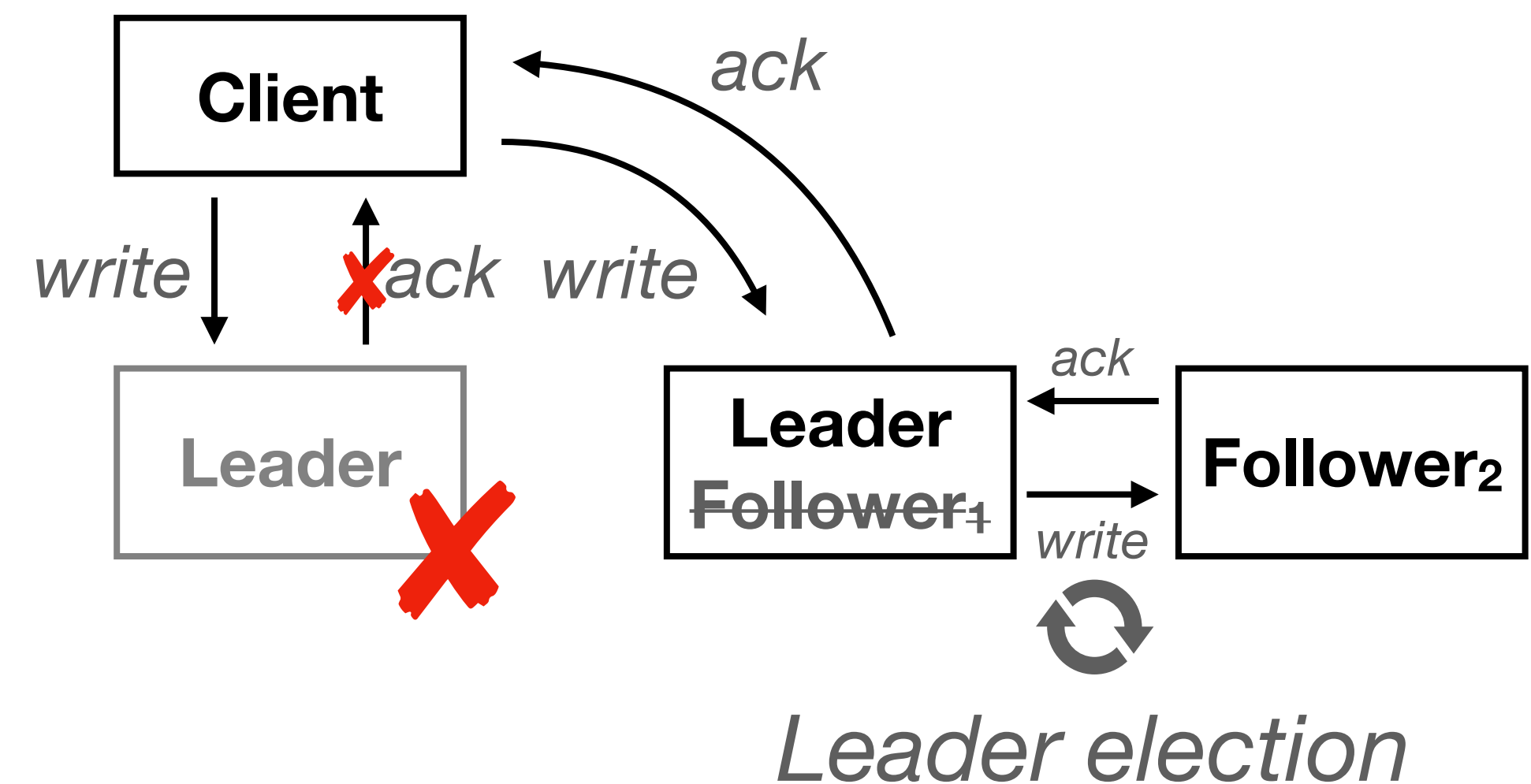
Data replication

Replication strategies: passive replication

- What if a *leader* fails?
 - A **new leader** is elected and all requests will be submitted to it.

- And if it occurs during a write?

- The write is considered invalid, and servers that were able to persist it rollback to the previous state*
- The client must submit the write request to the new *leader*
- What if a *follower* fails? What happens when a **new follower** is added to the group?



* Varies across strategies, use cases, and implementations.

Data replication

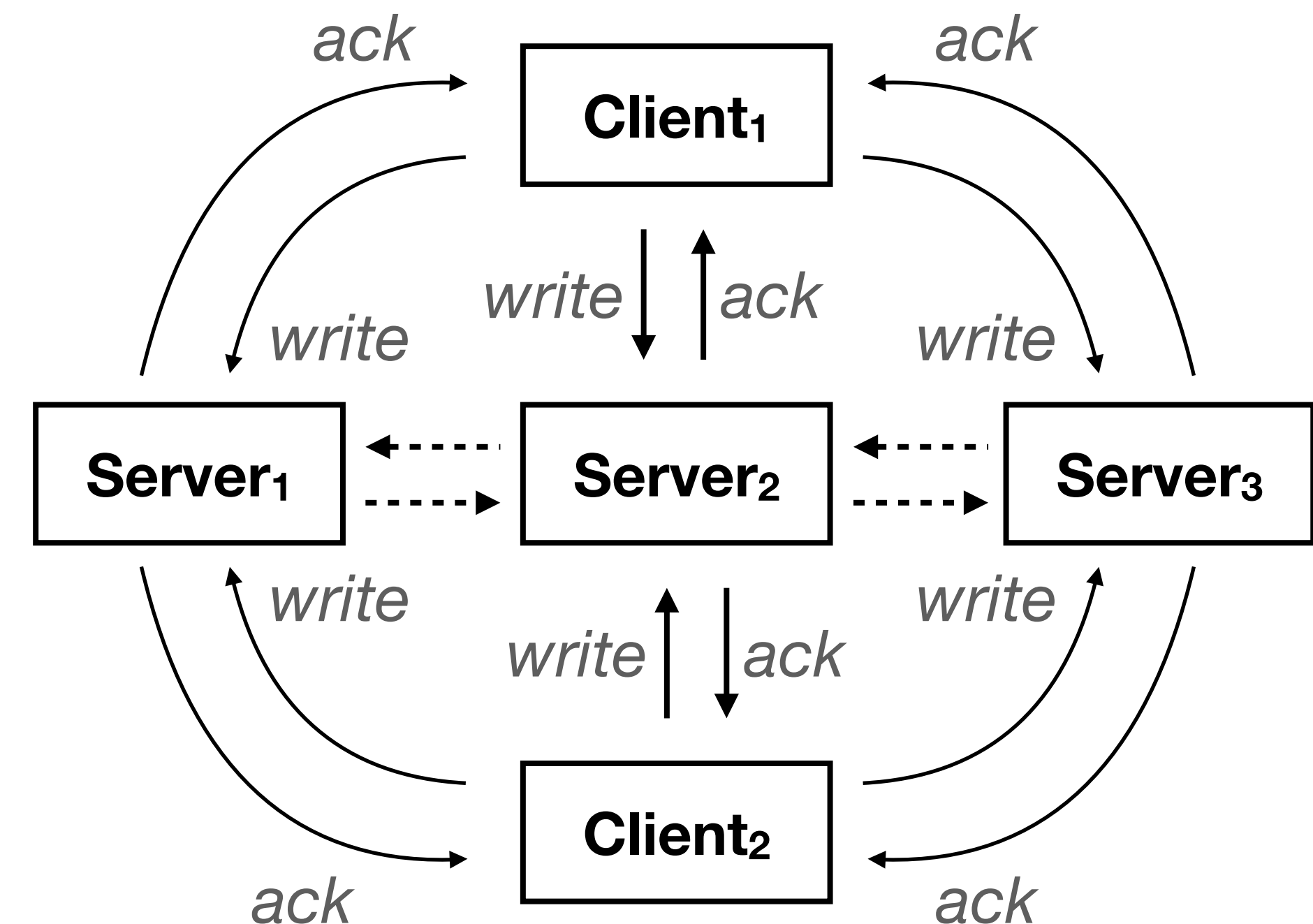
Replication strategies: passive replication

- **Simpler consistency management:** since only one node processes write requests, consistency is easier to maintain (single writer and total order)
- **Higher failover latency than active replication:** when the leader fails, a follower must be elected to take its place
- **Single point of contact:** the leader handles all write requests, potentially limiting scalability

Data replication

Replication strategies: active replication

- All servers process the same client requests simultaneously, ensuring that all nodes maintain an identical state
- Also known as *active-active replication*
 - The client broadcasts requests to all servers
 - Each server processes the request **independently** and **deterministically** to maintain consistency
 - If replicas communicate with each other (*leader/followers*), a coordination protocol (e.g., Paxos, Raft) ensures ordering and consistency
 - All replicas generate the same result, and the client receives a response



Data replication

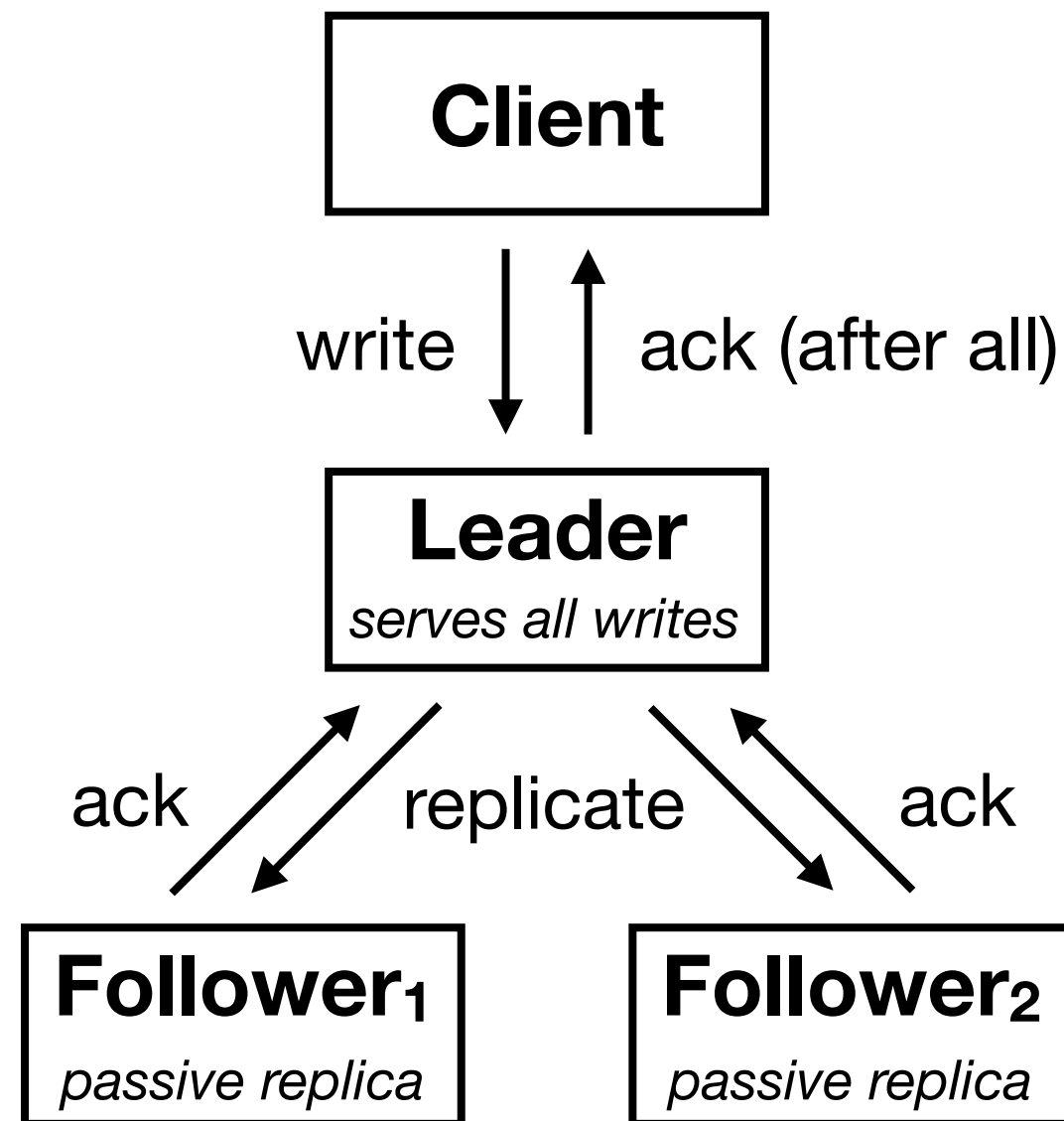
Replication strategies: active replication

- **High availability:** since all replicas are active, the system can tolerate failures without reconfiguration
- **Load balancing*:** Clients can query multiple servers, distributing the load
- **Inefficient under write-heavy workloads:** every write must be performed on all nodes, leading to redundant computation (e.g., hotspots)
- **Complex consistency management (conflict resolution):** ensuring identical execution across replicas requires coordination

Data replication

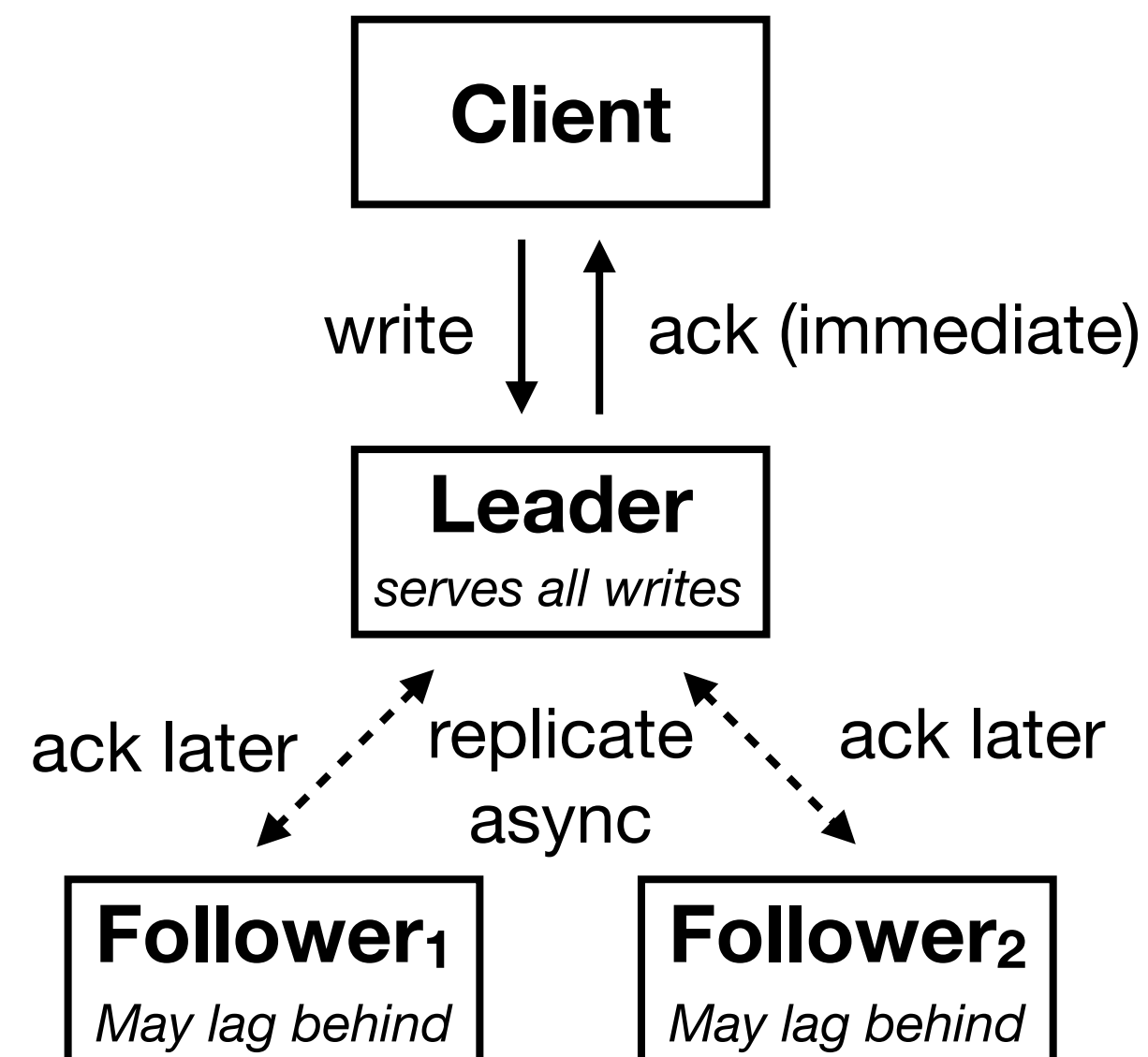
Combining all strategies

Synchronous + passive



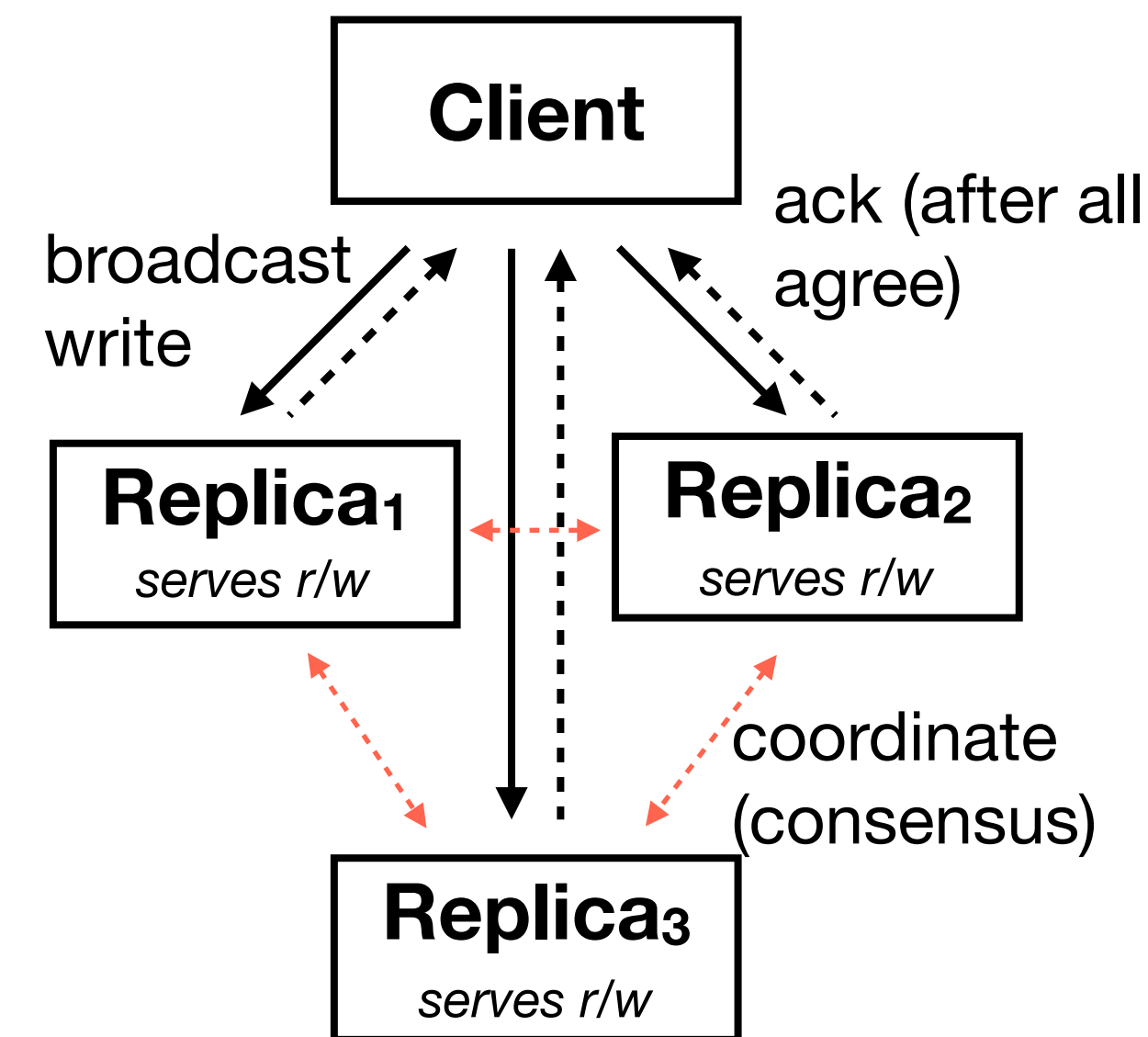
The client is blocked until both backups confirm. Zero data loss, but every write waits for the slowest replica.

Asynchronous + passive



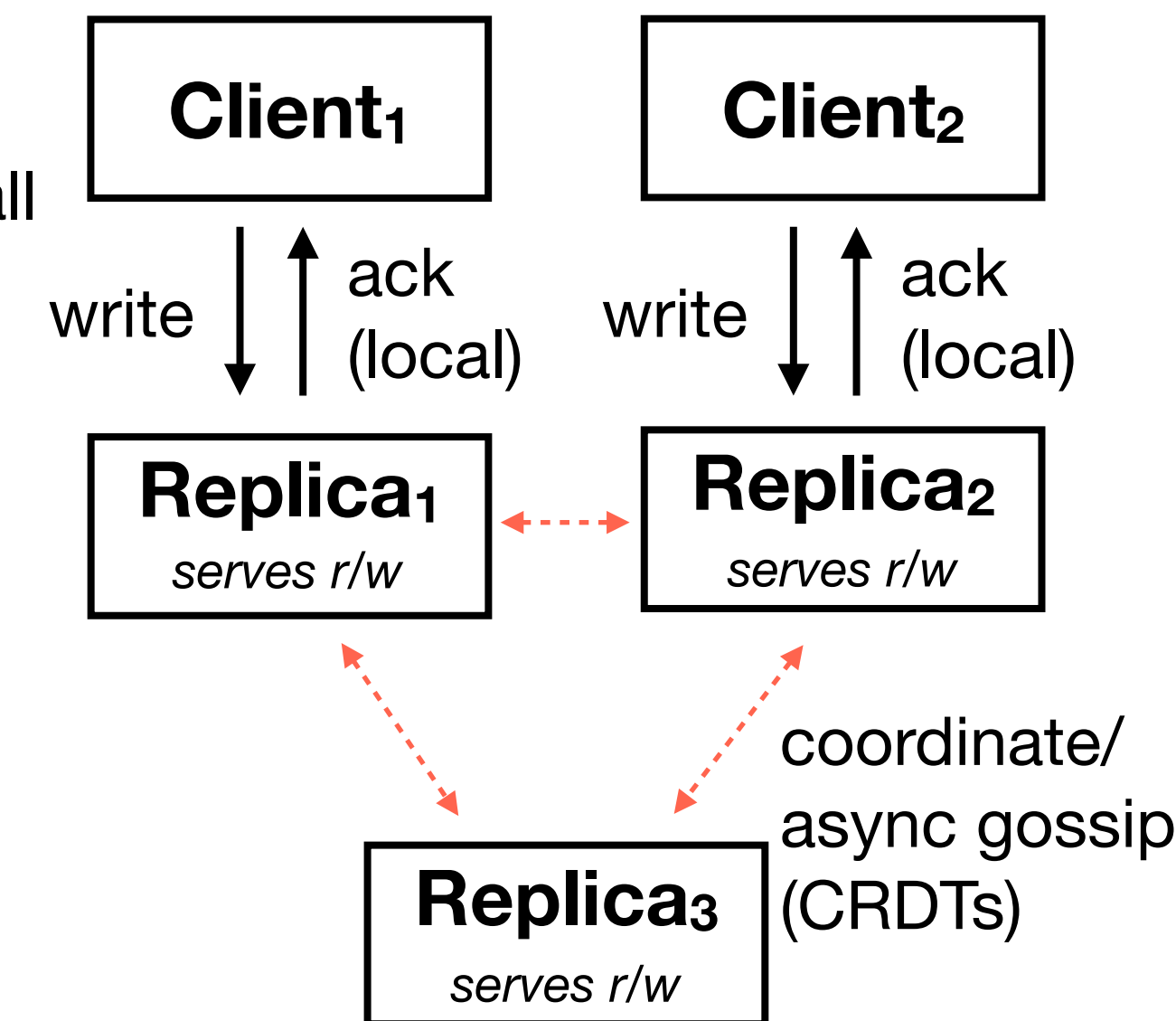
The client gets an immediate ACK from the leader; replication happens in background. Low latency, but a leader crash before replication completes means data loss.

Synchronous + active



The client broadcasts the write to all replicas, which must coordinate and all ACK before the client is confirmed. Maximum consistency and availability, but conflict resolution is mandatory and latency is high.

Asynchronous + active



Each client writes locally and gets an immediate ACK; replicas gossip updates to each other in background. Temporary divergence is expected and must be resolved (eventual consistency).

Data replication

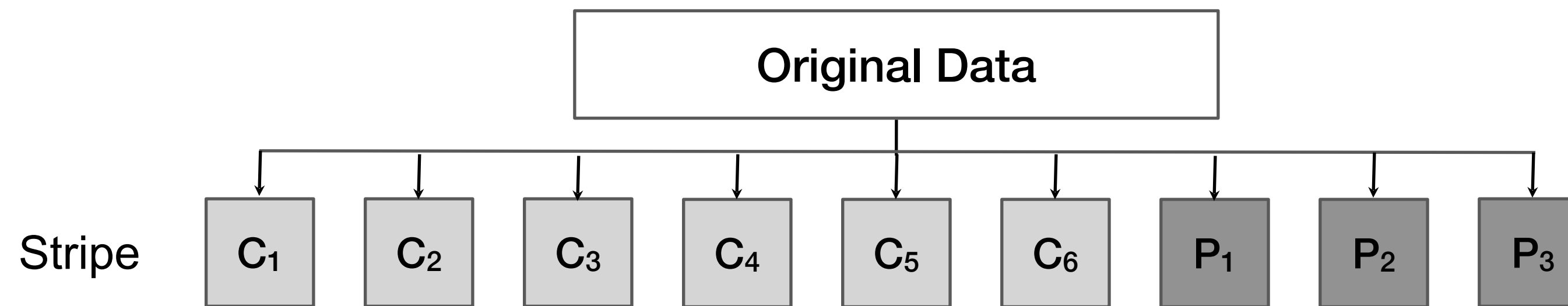
Geo-replication

- Data is stored in multiple **geographically distributed** data centers
- Used for **disaster recovery**, **low latency** (for global users), and **high availability**
- Replication can be
 - Synchronous or asynchronous
 - **Passive:** one active region for writes; simpler consistency but slower failover
 - **Active:** all regions accept reads and writes; complex conflict resolution (last-write-wins, CRDTs, ...)
- AWS S3, Google Cloud Spanner, Azure Cosmos DB

Erasure Codes

(forward error correction)

- Data is broken into chunks with redundant information that are then spread across several servers for availability and load-balancing purposes
- Main operations
 - **Encode:** divides original data into k fixed-size chunks and encodes them to form n ($k+m$) *coded chunks*
 - **Decode:** any k out of the n *coded chunks* can be decoded to reconstruct the original data
 - The collection of n *coded chunks* is referred to as a **stripe**
 - The chunks of a stripe can be distributed across n servers to tolerate up to **$n - k$ server failures**

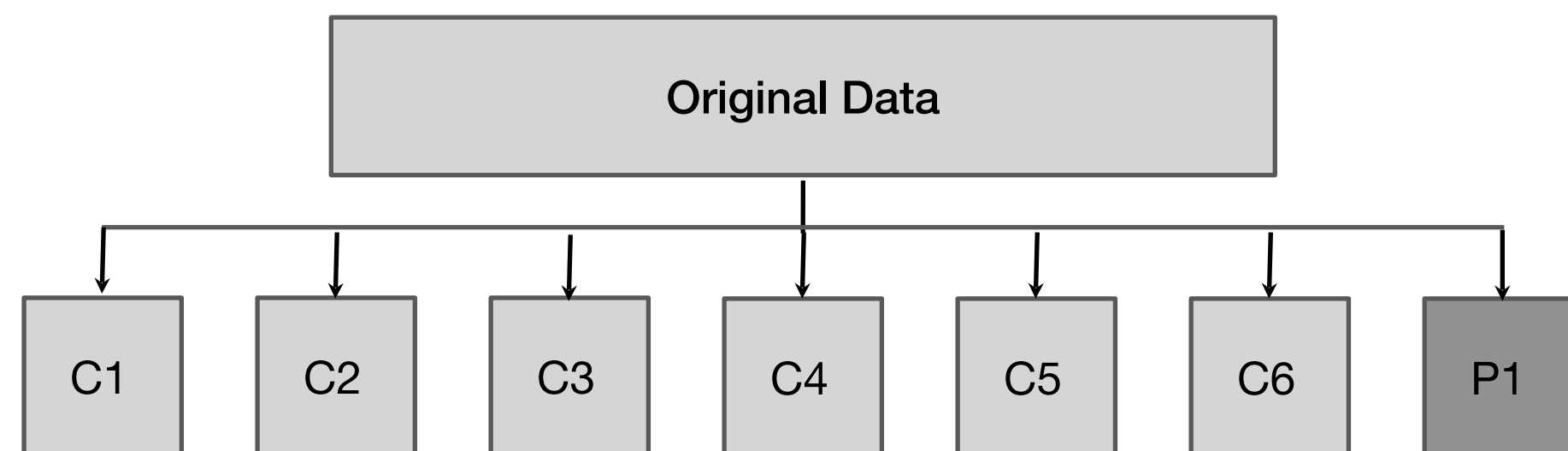


Original data is divided into a stripe with $k=6$ *chunks* and encoded into $n=9$ *coded chunks* ($m=3$ parity chunks)

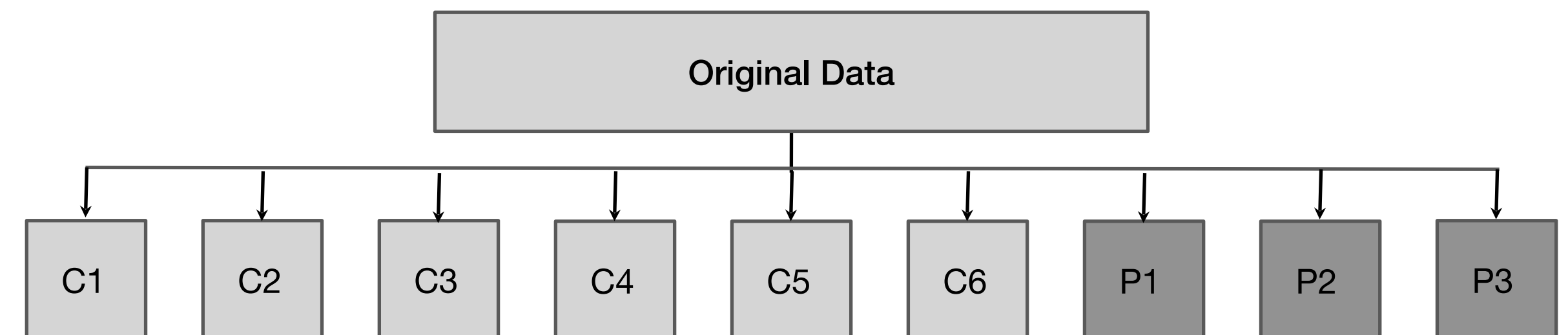
Erasure Codes (EC)

Two examples of codes

- ◎ **XOR:** k chunks are XORed with each other to produce *one* extra parity chunk
 - Efficient computation-wise
 - Only tolerates the loss (failure) of one chunk
 - Used in RAID 5
- ◎ **Reed-Solomon:** allows an *arbitrary* number of parity blocks to be generated from k chunks
 - Better fault tolerance
 - More expensive computational-wise



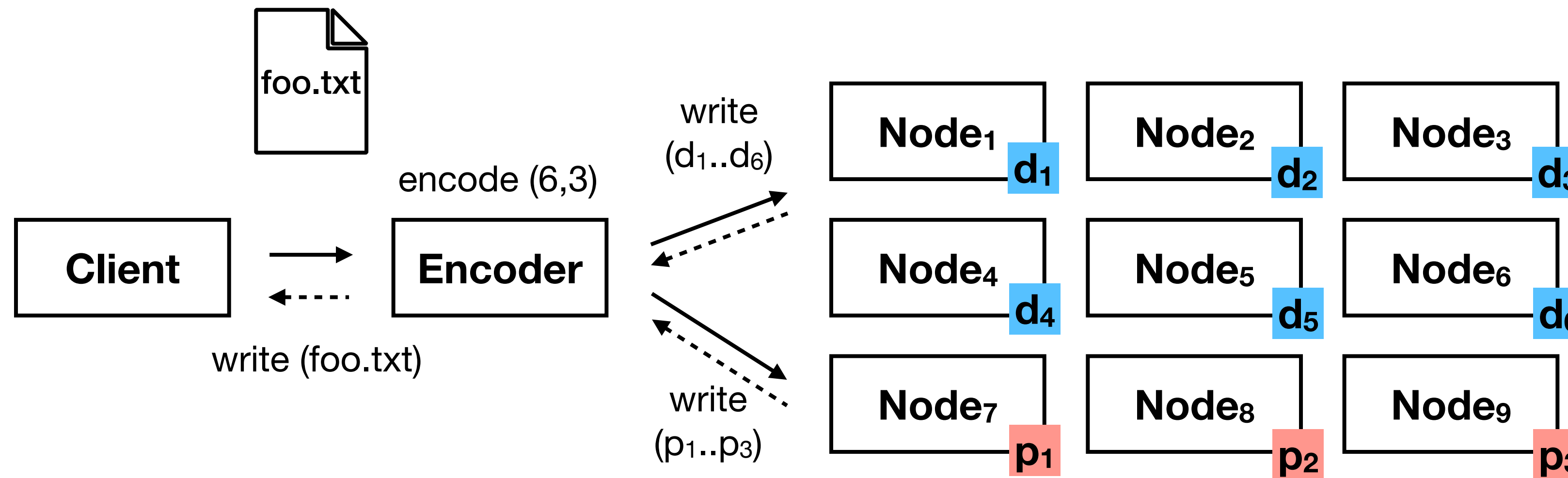
XOR erasure code with $k=6$ and $n=7$



Reed-Solomon erasure code with $k=6$ and $n=9$

Erasure Codes (EC)

Write path: Reed-Solomon (6,3)

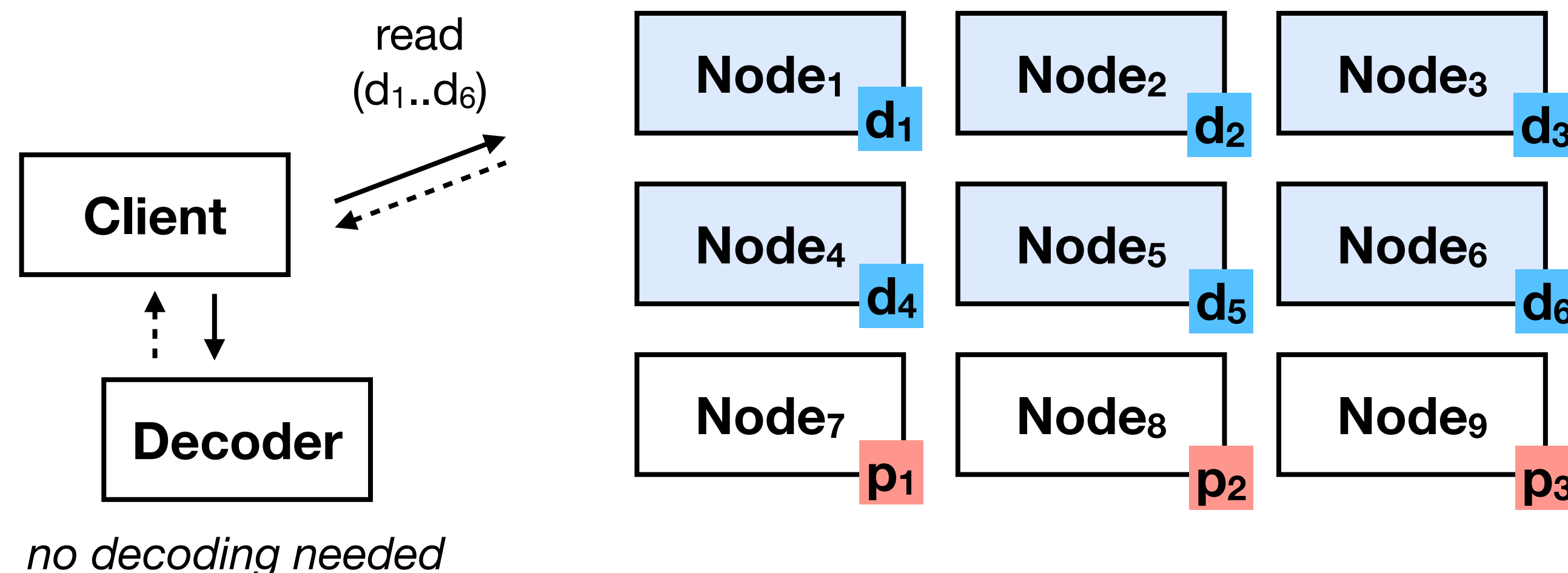


- Stripe the object into k data chunks (padding the last if needed)
- Encode the m parity chunks
- Scatter all $k+m$ chunks to distinct nodes
- Acknowledge once a quorum confirms

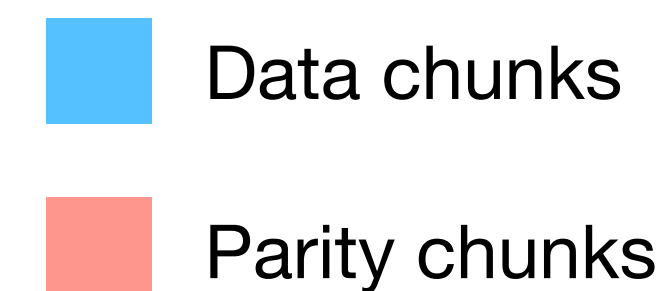
■ Data chunks
■ Parity chunks

Erasure Codes (EC)

Read path (normal read): Reed-Solomon (6,3)

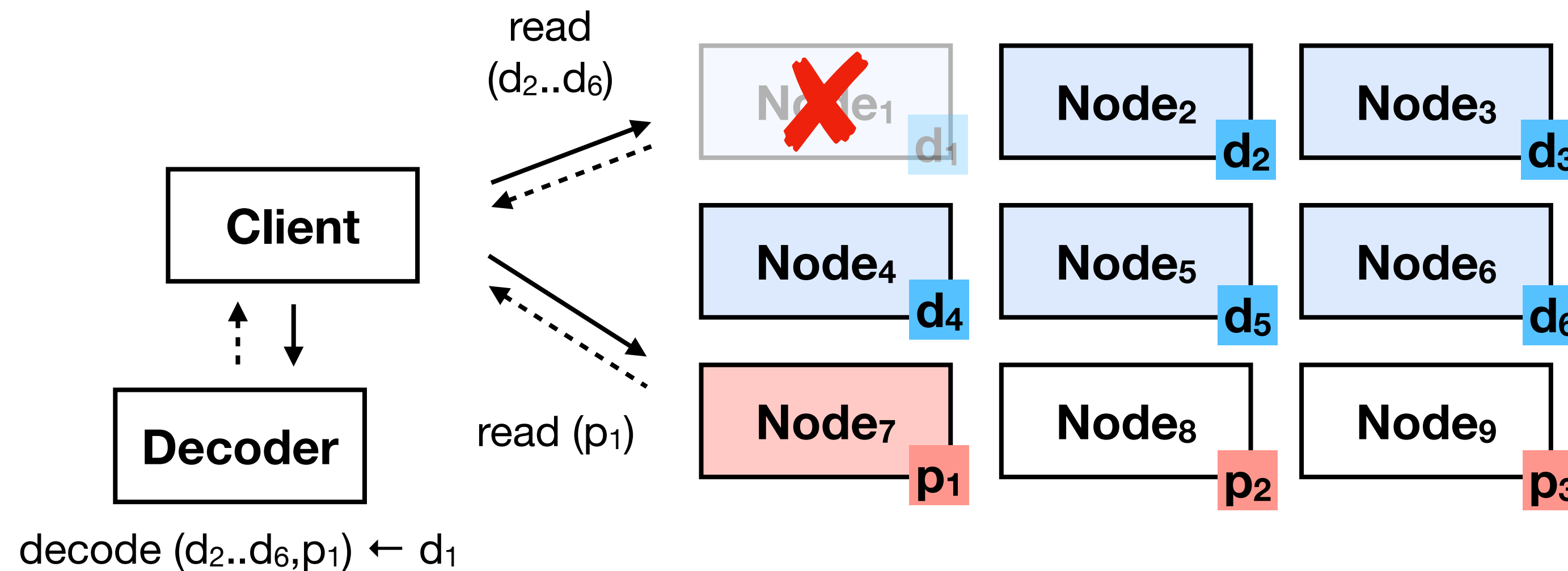


- Contact any k nodes holding data chunks
- Read chunks in parallel
- If all k data chunks are available, no decoding is needed
- Latency is bounded by the slowest of the k contacted nodes

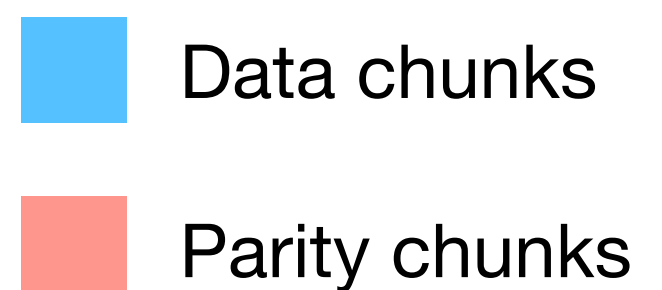


Erasure Codes (EC)

Read path (degraded read): Reed-Solomon (6,3)

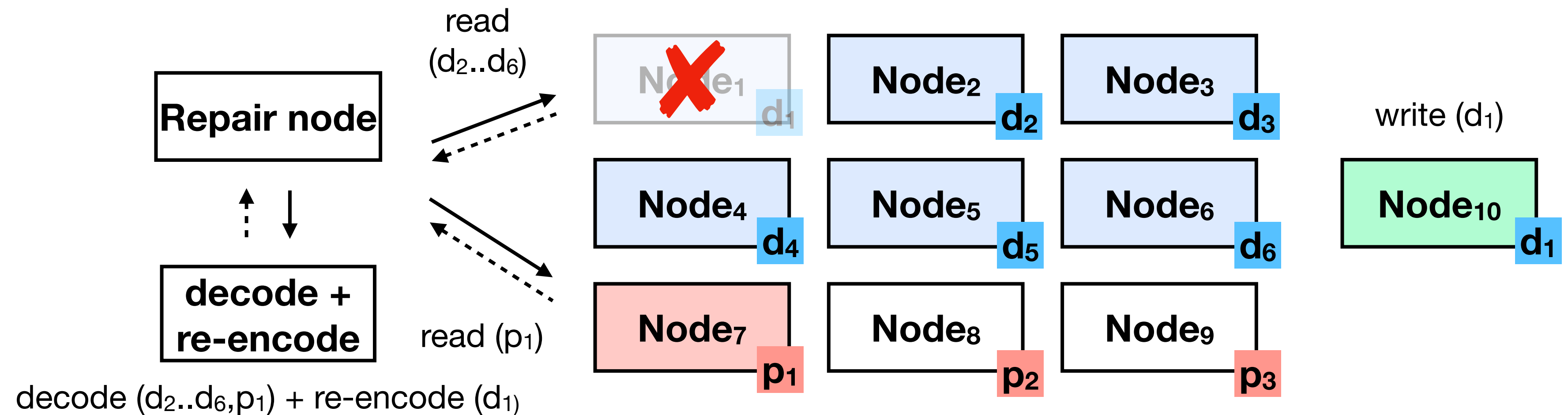


- Contact enough surviving nodes to gather any k chunks (data + parity)
- Reconstruct the missing data (decode)
- Return the result to the client
- Degraded reads are significantly more expensive (required reading more chunks and decoding)



Erasure Codes (EC)

Repair path: Reed-Solomon (6,3)



- Read any k surviving chunks from k nodes
- Decode the full stripe
- Re-encode the missing chunk
- Write it to a replacement node

■ Data chunks
■ Parity chunks

Erasure Codes

vs. replication

● **Advantages:** when compared to full replication, erasure codes

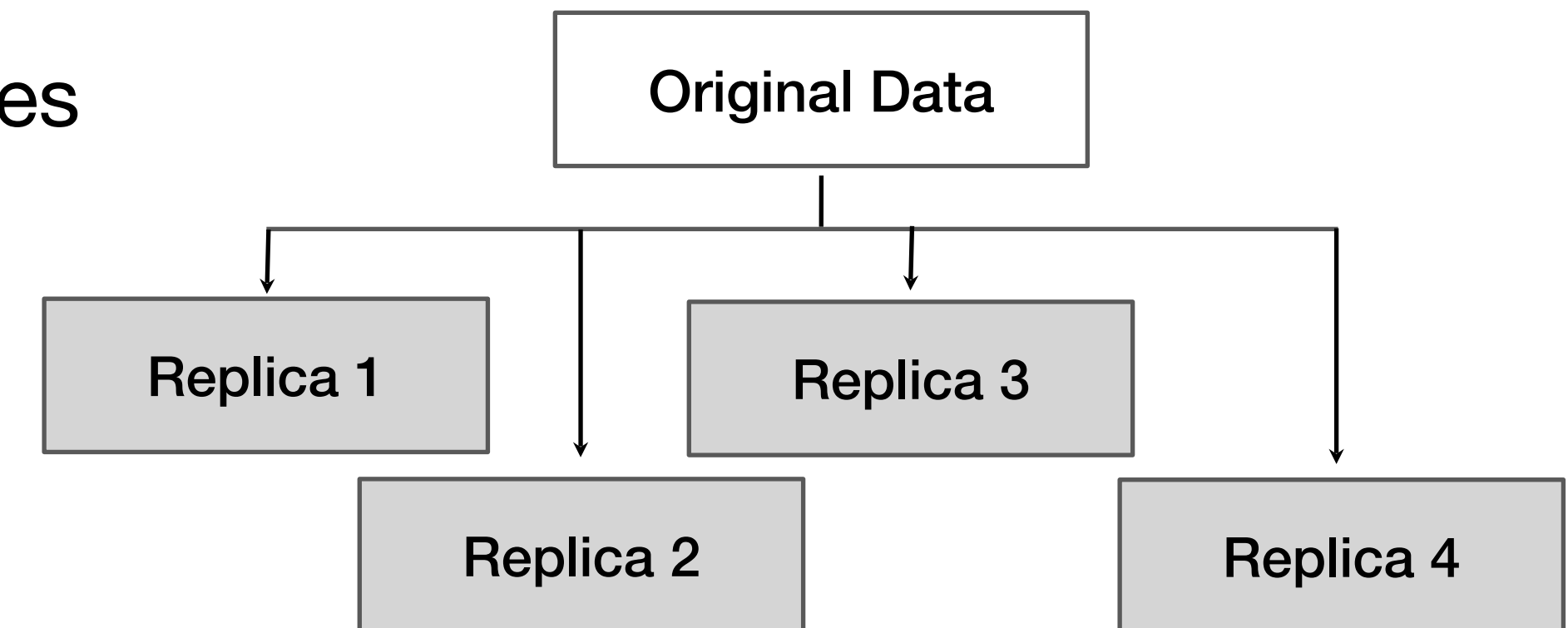
- ▶ have **comparable protection** against failures
- ▶ significantly **reduce storage space** overhead (costs)

● **Disadvantages:** when compared to full replication, erasure codes

- ▶ require **more servers** to store chunks and ensure identical fault tolerance guarantees to full replication
- ▶ have **higher computational** demand (*i.e.*, encoding and decoding operations)
- ▶ **Failure repairing** and **data updates** require extra computation and network I/O

● New generations of erasure codes tackle these challenges

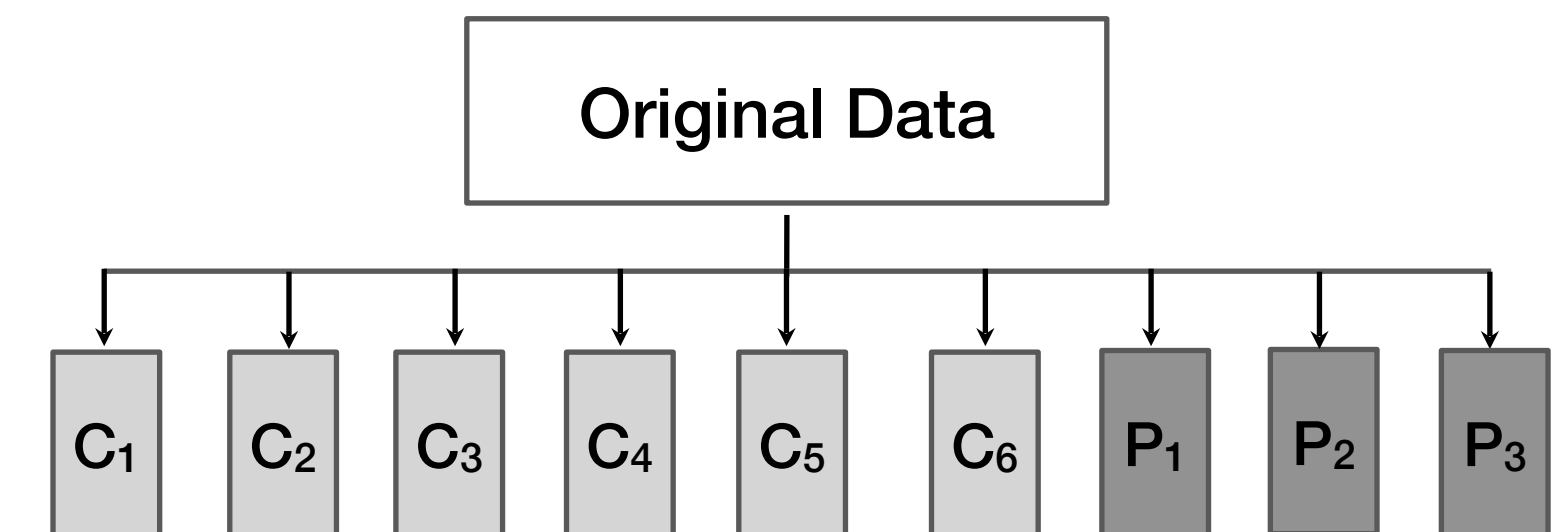
- ▶ Regenerating codes, LRCs, Piggybacking codes



Replication factor of 4

- ▶ Tolerates 3 failures
- ▶ 4x storage overhead

.....



Erasure codes ($k = 6, n = 9$)

- ▶ Tolerates 3 failures
- ▶ 1.5x storage overhead

More Information

- **Chapters 3 and 4** - Tom White. **Hadoop: The Definitive Guide** - O'Reilly, 5th Edition.
- **Chapter 5** - Martin Kleppmann. **Designing Data-Intensive Applications**. O'Reilly, 2nd Edition.
- **A Comprehensive Study of the Past, Present, and Future of Erasure Coding for Storage Systems** - Shen *et al.*, ACM Transactions on Storage, 2025.